

1. A class consists of $n=24$ students. The average BMI for the class is 32.46, and the standard deviation is 6.92. Six new students have joined the class. Their BMIs are [23.5, 30.5, 42.3, 34.5, 49.5, 44.2]. Write Python code and calculate the sample mean and standard deviation for the 30 students using the recursive formula.

```
from math import sqrt

new_bmis = [23.5, 30.5, 42.3, 34.5, 49.5, 44.2]
sample_mean, sample_std, n_students = 32.46, 6.92, 24
Q = sample_std ** 2 * n_students
while new_bmis:
    n_students += 1
    bmi_i = new_bmis.pop(0)
    last_sample_mean = sample_mean
    sample_mean = last_sample_mean + ((bmi_i - last_sample_mean) / float(n_students))
    Q = Q + (bmi_i - (last_sample_mean)) * (bmi_i - sample_mean)

! finish the standard deviation equation outside of the do loop
sample_std = sqrt(Q / float(n_students))
print("sample mean:", sample_mean)
print("sample standard deviation:", sample_std)
```

```
/bin/bash: line 1: finish: command not found
sample mean: 33.45133333333333
sample standard deviation: 7.600638014155273
```

2. The probability that a student owns a microwave oven is 0.75, and that a student owns a TV is 0.25. The probability that a student owns both a microwave and a TV is 0.16. Find the probability that a student owns a microwave or a TV, but not both.

```
print("probability that a student owns a microwave or a TV, but not both:")
p_microwave, p_tv, p_both = 0.75, 0.25, 0.16
print(p_microwave + p_tv - p_both)
```

```
probability that a student owns a microwave or a TV, but not both:
0.84
```

3. Five cards are drawn from a standard deck of cards without replacement. Find the probability of getting

```
from math import floor, log10

def sigfigs(x:float | int, n_digits:int, /) -> float:
    return round(x, n_digits - int(floor(log10(abs(x)))) - 1)
```

- a. All red cards

```
from functools import reduce

num_remaining_in_red = 26
num_cards = 52
top = reduce(lambda x, y: x*y, [num_remaining_in_red - num_cards_drawn for num_cards_drawn in range(5)])
bottom = reduce(lambda x, y: x*y, [num_cards - num_cards_drawn for num_cards_drawn in range(5)])

print(f"{sigfigs(top/bottom*100, 3)}%")
```

```
2.53%
```

- b. All diamonds

```
from functools import reduce
```

```

num_remaining_in_red = 13
num_cards = 52
top = reduce(lambda x, y: x*y, [num_remaining_in_red- num_cards_drawn for num_cards_drawn in range(5)])
bottom = reduce(lambda x, y: x*y, [num_cards- num_cards_drawn for num_cards_drawn in range(5)])

print(f"{sigfigs(top/bottom, 3)*100}%")

```

↔ 0.0495%

c. All aces

0%, there are not five aces

✓ Find the probabilities of the following events that the person is

a. P(Over 40)

680/1,000 = 68%

b. P(Mild and Over 40)

121/1,000 = 12.1%

c. P(Mild or Over 40)

if this is an exclusive or, (51+680-121)/1,000 = 61%. Otherwise, (680+51)/1,000 = 73.1%

d. P(not Mild)

(1000-172)/1,000 = 82.8%

e. P(Mild | Over 40)

121/172 ≈ 70.3%

✓ 5. At a particular University, 1/2 of the students drink alcohol and 1/3 of the students smoke cigarettes.

(a) What is the largest possible fraction of students who do neither?

1/2

(b) It turns out that, in fact, 1/3 of the students do neither. What fraction of the students does both?

1/6

✓ You shuffle a standard deck of cards, then draw four cards.

✓ (a) What is the probability all four are the same suit?

```

from functools import reduce

```

```

num_remaining_in_suit = 12
num_cards = 51
top = reduce(lambda x, y: x*y, [num_remaining_in_suit- num_cards_drawn for num_cards_drawn in range(3)])
bottom = reduce(lambda x, y: x*y, [num_cards- num_cards_drawn for num_cards_drawn in range(3)])

print(f"{sigfigs(top/bottom, 3)*100}%")

```

↔ 1.06%

- ✓ (b) What is the probability all four are red?

```
from functools import reduce

num_remaining_in_red = 26
num_cards = 52
top = reduce(lambda x, y: x*y, [num_remaining_in_red- num_cards_drawn for num_cards_drawn in range(4)])
bottom = reduce(lambda x, y: x*y, [num_cards- num_cards_drawn for num_cards_drawn in range(4)])

print(f"{sigfigs(top/bottom, 3)*100}%")
```

↔ 5.52%

- ✓ (c) What is the probability each has a different suit?

```
from functools import reduce
num_cards = 52
top = reduce(lambda x, y: x*y, [num_cards- (num_cards_drawn*13) for num_cards_drawn in range(4)])
bottom = reduce(lambda x, y: x*y, [num_cards- num_cards_drawn for num_cards_drawn in range(4)])

print(f"{sigfigs(top/bottom, 3)*100}%")
```

↔ 10.5%

- ✓ You shuffle a standard deck of playing cards, and deal a hand of 10 cards. With what probability does this hand have five red cards?

First of all, this question NEEDS clarification. Is the question asking the chances of getting exactly five red cards or at least five red cards? The formula for finding the chances of exactly five red cards is as follows:

The total number of ways to draw 10 cards out of 52 is $C(52, 10)$. The number of ways to draw 5 red cards out of 26 is $C(26, 5)$. The number of ways to draw the remaining 5 cards out of the 26 non-red cards is $C(26, 5)$.

The probability is therefore:

$$P = [C(26, 5) * C(26, 5)] / C(52, 10)$$

If we just need to draw AT LEAST five cards, we would have to do extra calculations, shown below

```
from math import comb

exactly_5_cards = comb(26, 5) * comb(26, 5) / comb(52, 10)
print(f"exactly 5 red cards: {sigfigs(exactly_5_cards*100, 3)}%")
```

↔ exactly 5 red cards: 27.4%

```
at_least_5_cards = 0
for num_red_needed in range(5, 11):
    at_least_5_cards += comb(26, num_red_needed) * comb(26, 10 - num_red_needed) / comb(52, 10)
print(f"at least 5 red cards: {sigfigs(at_least_5_cards*100, 3)}%")
```

↔ at least 5 red cards: 63.7%

